Value Iteration and Q Learning

7 Post

CS 3244
Machine Learning

NUS National University of Singapore | Computing

Recap and Forecast

Understand how validation and regularization complement to address overfitting:

- **Regularization:** a means of restraining the model.
  - L^p norm forms of regularization.

$J_{test}(h) = J_{train}(h) +$ overfit penalty

- **Validation**, as a reality check by peeking (at the bottom line).
  - Cross Validation variants: k-fold or leave one out.

Learning Outcomes for this week:

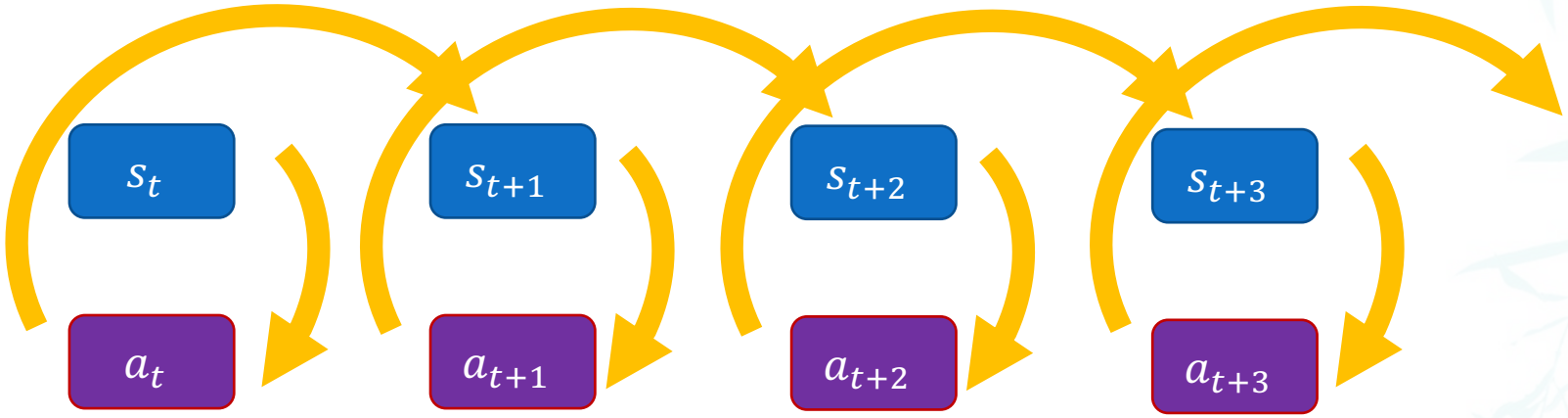
- Understand the basic terminology of RL: Policy, reward, action, observation and (environment, agent) model.
- Relate how a **Markov Decision Process** (MDP) embodies RL concepts.
- Understand the **Bellman equations** as a recursive form for calculating future rewards and state values.
- Solve planning and RL problems using the **Value Iteration** and **Q Learning** algorithms.
- Differentiate Value Iteration for planning from Q Learning for RL by their modeling assumptions

Dissecting the MDP

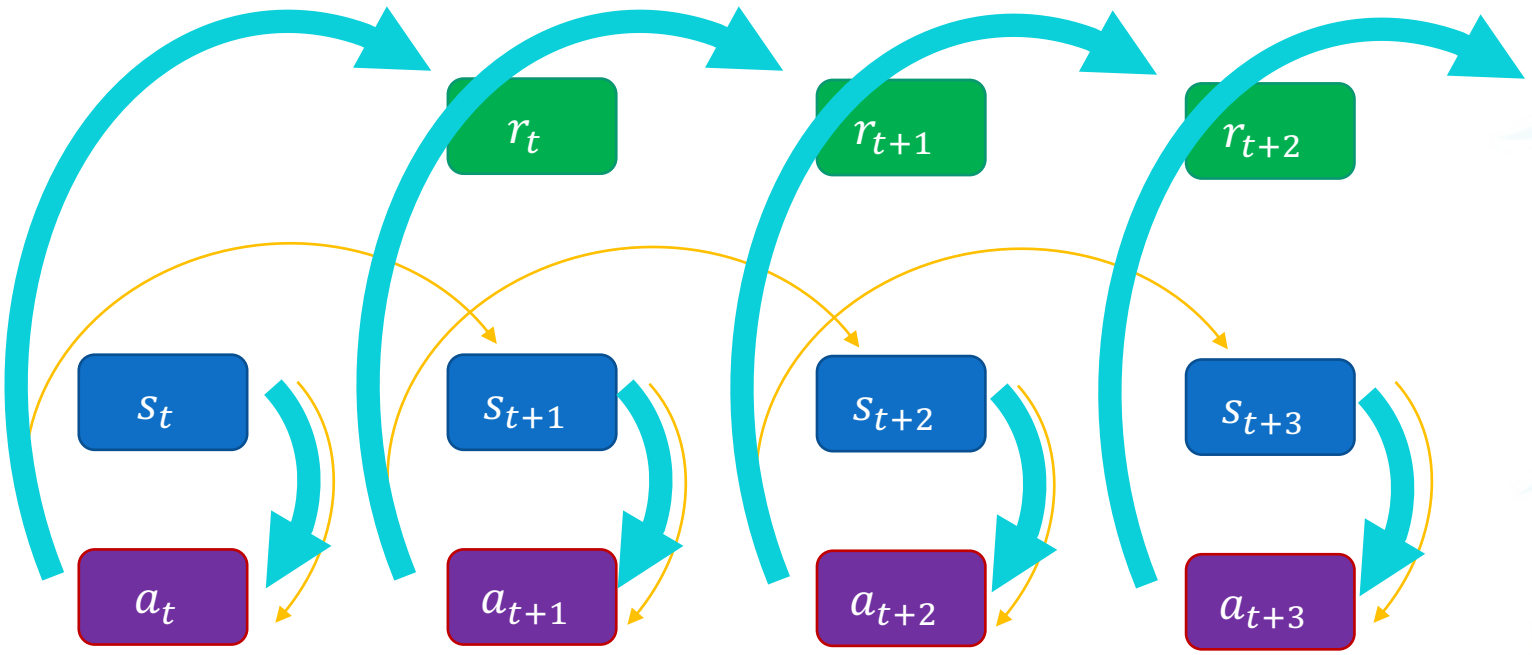
CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

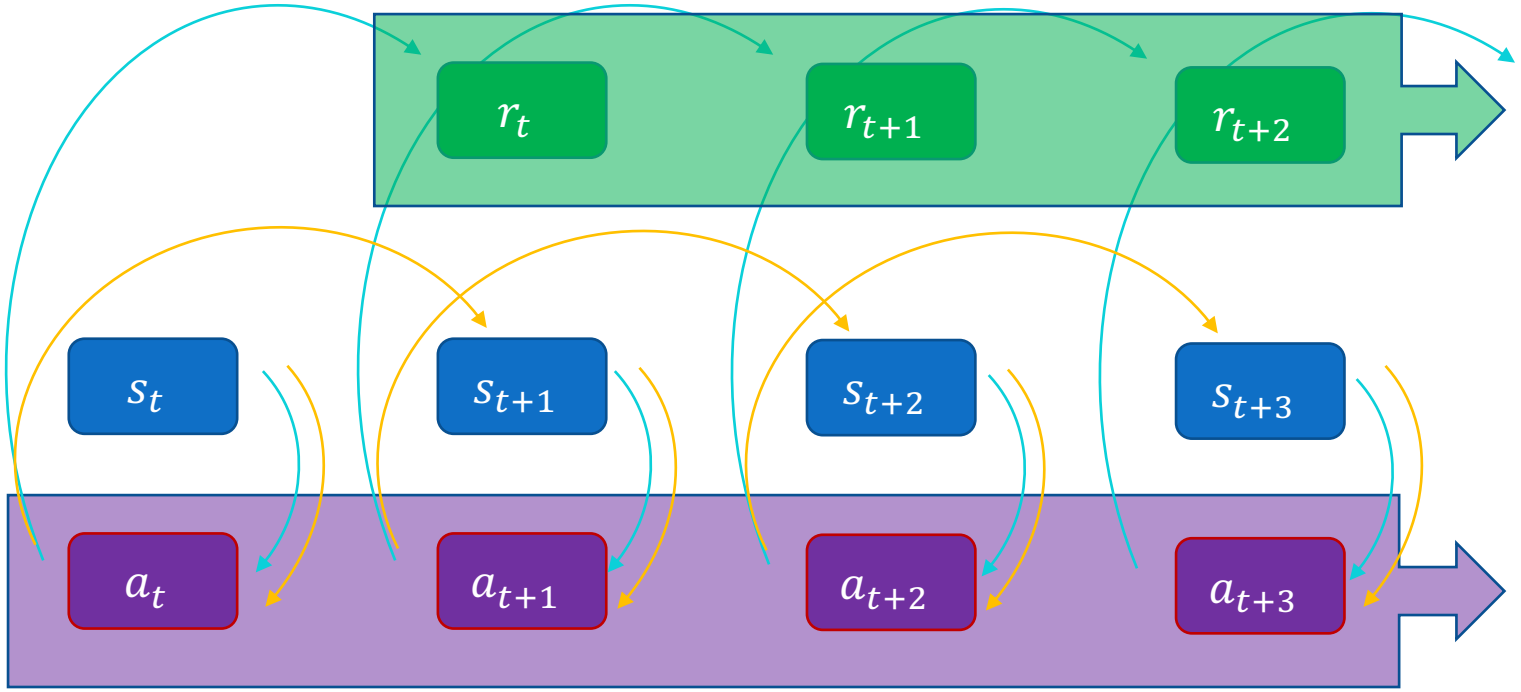
MDP Iterative Formulation



Reward – Iterative formulation



Agent's Actions and Value Estimation



Value function computed based on rewards

Actions chosen based on policy

Return (aka Utility)

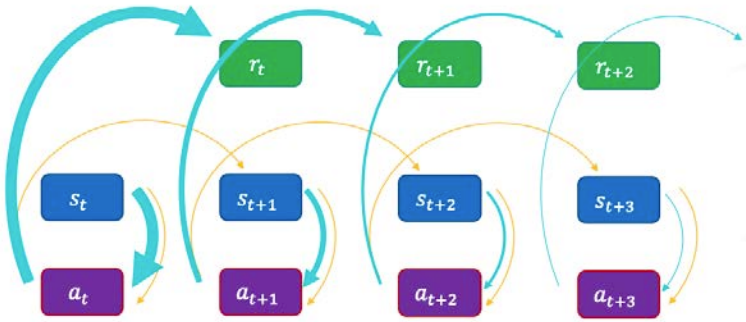
The return g_t is the total discounted reward from time step t.

$$g_t = r_{t+1} + \gamma r_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

- The discount $\gamma \in [0, 1]$ is the present value of future rewards.
- The value of receiving reward r after $k + 1$ time-steps is $\gamma^k r$.
- This values immediate reward above delayed reward.
  - γ close to 0 leads to "near-sighted" evaluation
  - γ close to 1 leads to "far-sighted" evaluation

Why Discount?

Most Markov reward and decision processes are discounted. Why?



- Mathematically convenient to discount rewards
- Avoids infinite returns from cycles
- Uncertainty about the future
- Animal/human behaviour shows preference for immediate reward
- If the reward is financial, immediate rewards may earn more interest than delayed rewards
- It is sometimes possible to use *undiscounted* Markov reward processes (i.e. $\gamma = 1$), if all sequences terminate.

Value Function

The value function $v(s)$ gives the long-term value of state s.

The state value function $v(s)$ of an MRP is the expected return starting from state s:

$$v(s) = \mathbb{E}(g_t | s_t = s)$$

Policies

A policy π is a distribution over actions given states,

$$\pi(a|s) = P(a_t = a|s_t = s)$$

- A policy fully defines the behavior of an agent
- MDP policies depend on the current state (not the history)
  i.e. Policies are stationary (time-independent):

$$a_t \sim \pi(\cdot\,|s_t), \forall t > 0$$

Value Functions of an MDP

- The *state–value function* $v_\pi(s)$ of an MDP is the expected return starting from state s, and then following policy π:

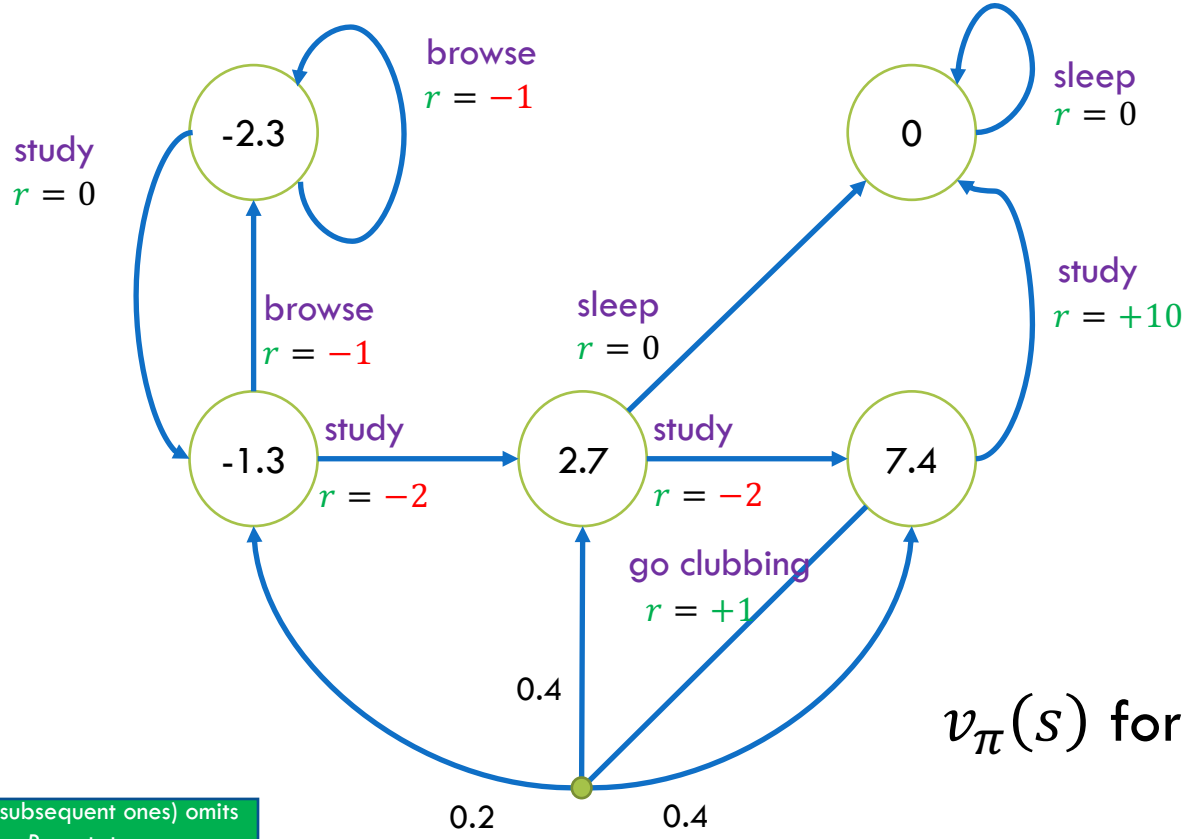
$$v_\pi(s) = \mathbb{E}_\pi(g_t \mid s_t = s)$$

- The *action–value function* $q_\pi(s, a)$ is the expected return starting from state s, taking action a, and then following policy π:

'Q' for Quality

$$q_\pi(s, a) = \mathbb{E}_\pi(g_t \mid s_t = s, a_t = a)$$

Example State–Value Function



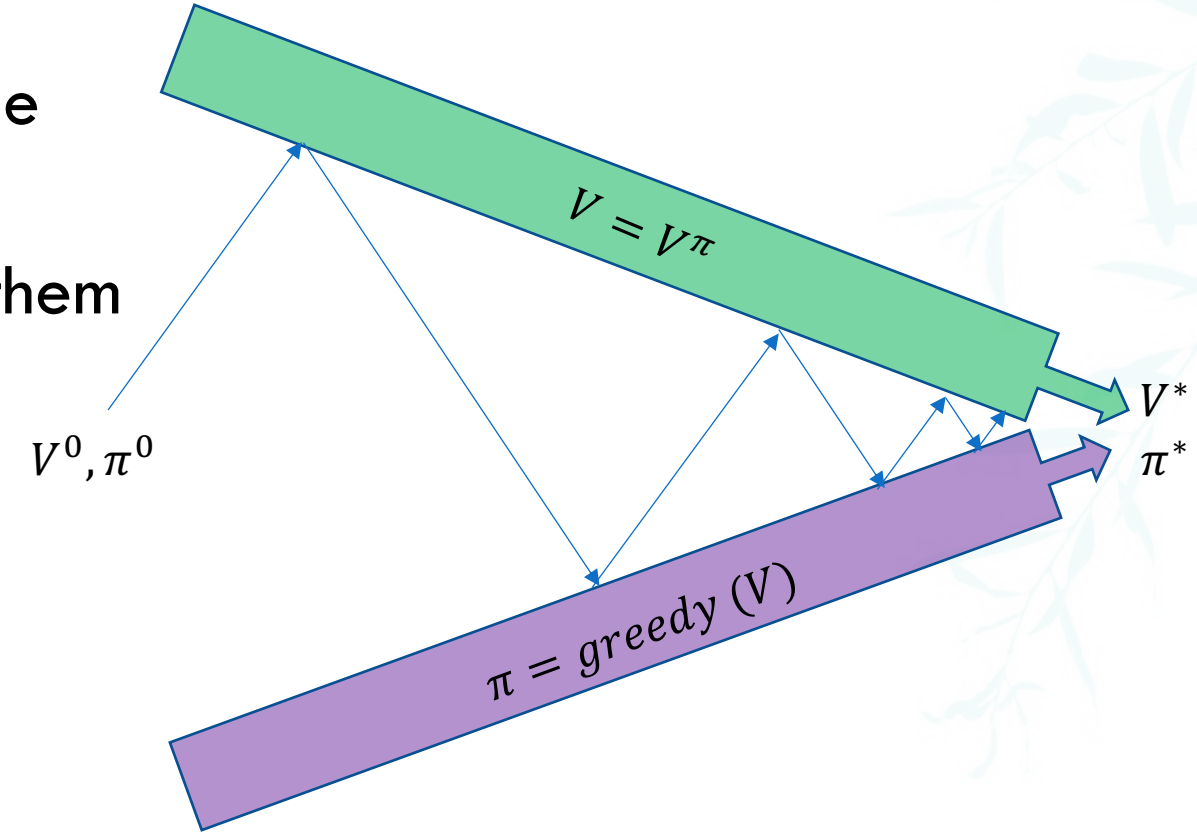
$v_\pi(s)$ for $\pi(a|s) = 0.5, \gamma = 1$

This diagram (and subsequent ones) omits the earlier *Pass* state

Generalised Policy Iteration

Start with a random value function and policy

And iteratively improve them



Bellman Expectation

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Bellman Expectation Equation

The state–value function can be decomposed into immediate reward plus discounted value of successor state:

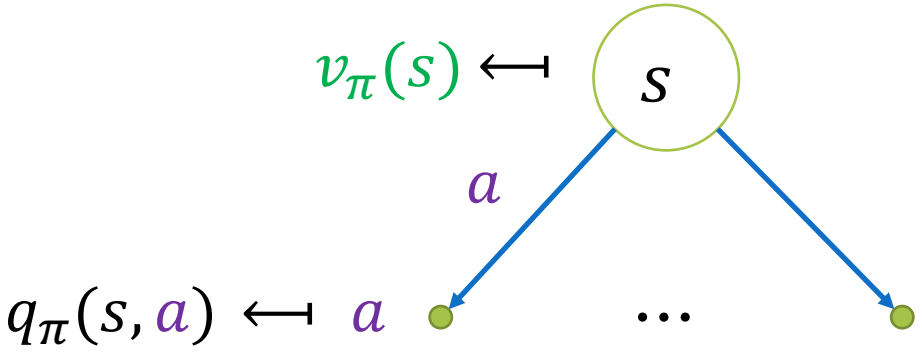
$$v_\pi(s) = \mathbb{E}_\pi\left(\mathcal{R}_{t+1} + \gamma v_\pi(s_{t+1}) | s_t = s\right)$$

The action–value function can similarly be decomposed:

$$q_\pi(s, a) = \mathbb{E}_\pi\left(\mathcal{R}_{t+1} + \gamma q_\pi(s_{t+1}, a_{t+1}) | s_t = s, a_t = a\right)$$

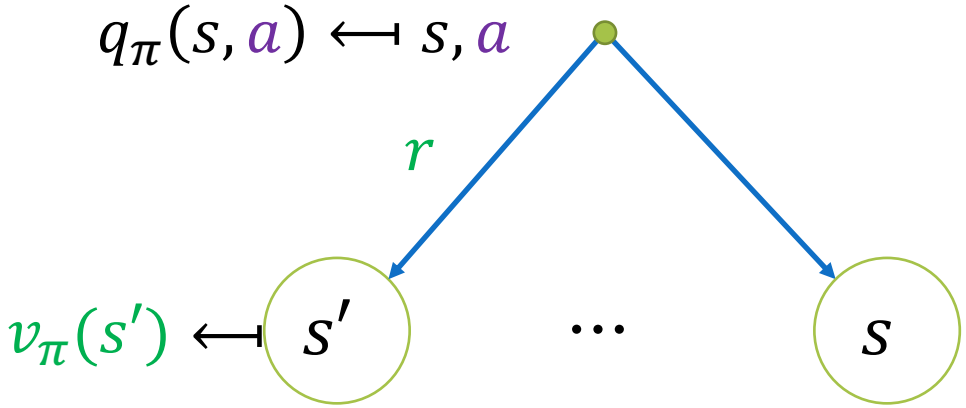
Bellman Expectation Equation for v_π

Bellman Backup (Diagram): Calculate the value function by recursing through one step.



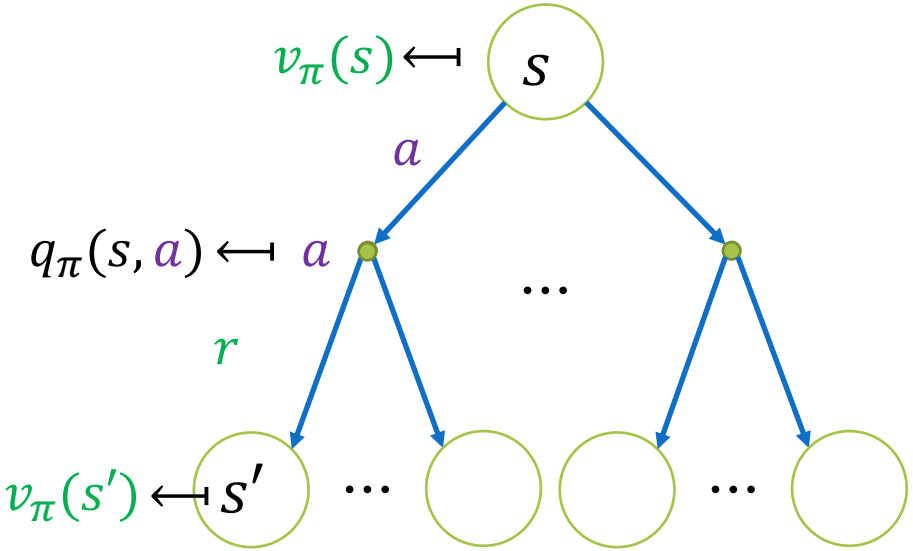
$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s)\, q_\pi(s,a)$$

Bellman Expectation Equation for q_π



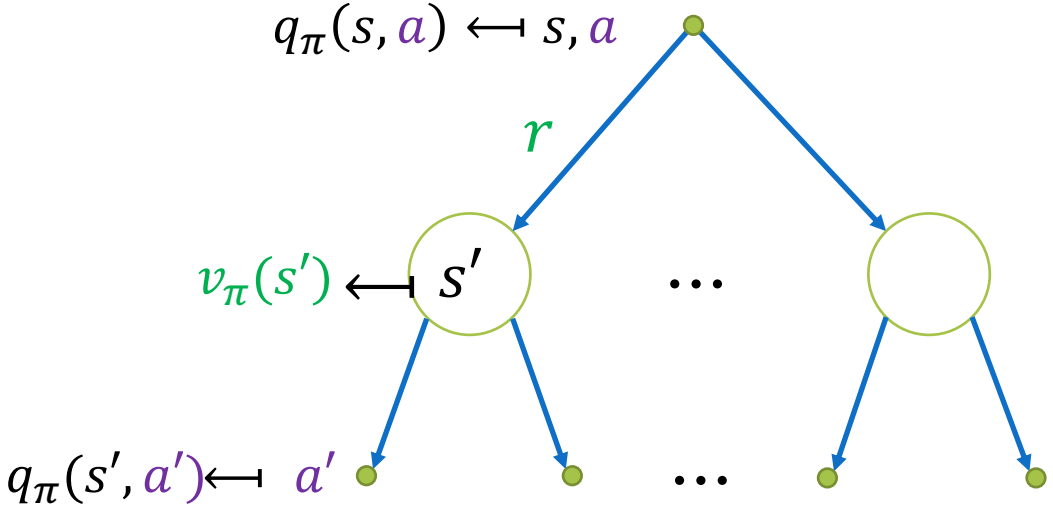
$$q_\pi(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{s,s'}^a \, v_\pi(s')$$

Recursive Formulation for v_π



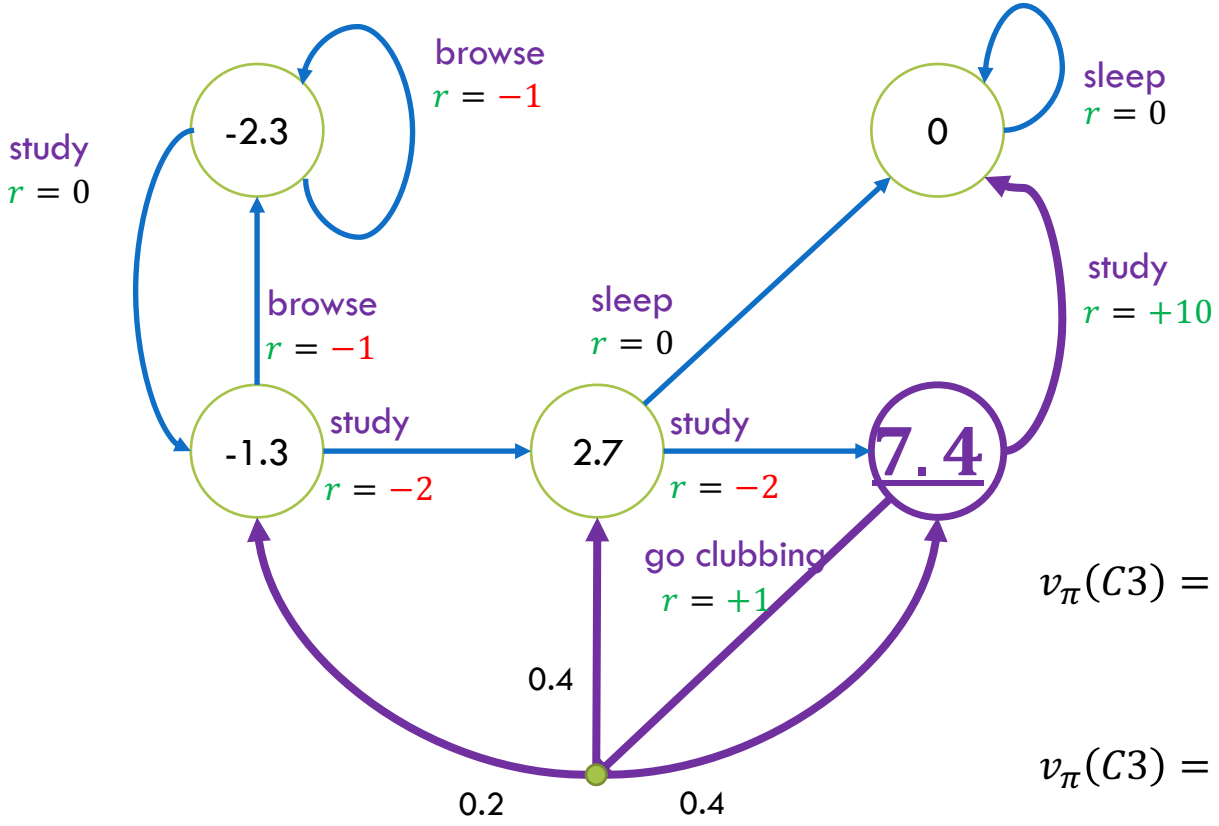
$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \underbrace{\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{s,s'}^a \, v_\pi(s')}_{q_\pi(s,a)}$$

Recursive Formulation for q_π



$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{s,s'}^a \underbrace{\sum_{a' \in \mathcal{A}} \pi(a'|s')\, q_\pi(s',a')}_{v_\pi(s')}$$

Bellman Expectation for Student MDP



Under a uniform policy

$$v_\pi(C3) = \frac{1}{2}(10) + \frac{1}{2}(1 + 0.2\times -1.3 + 0.4\times 2.7 + 0.4\times 7.4)$$

$$v_\pi(C3) = 7.4$$

Bellman Expectation Equation (Matrix Form)

The Bellman expectation equation can be expressed concisely using the induced MRP:

$$v_\pi = \mathcal{R}^\pi + \gamma \mathcal{P}^\pi v_\pi$$

with the direct solution:

$$v_\pi = (I - \gamma \mathcal{P}^\pi)^{-1} \mathcal{R}^\pi$$

Optimal Value Functions

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Optimal Value Function

The optimal state-value function $v_*(s)$ is the maximum state–value function over all policies:

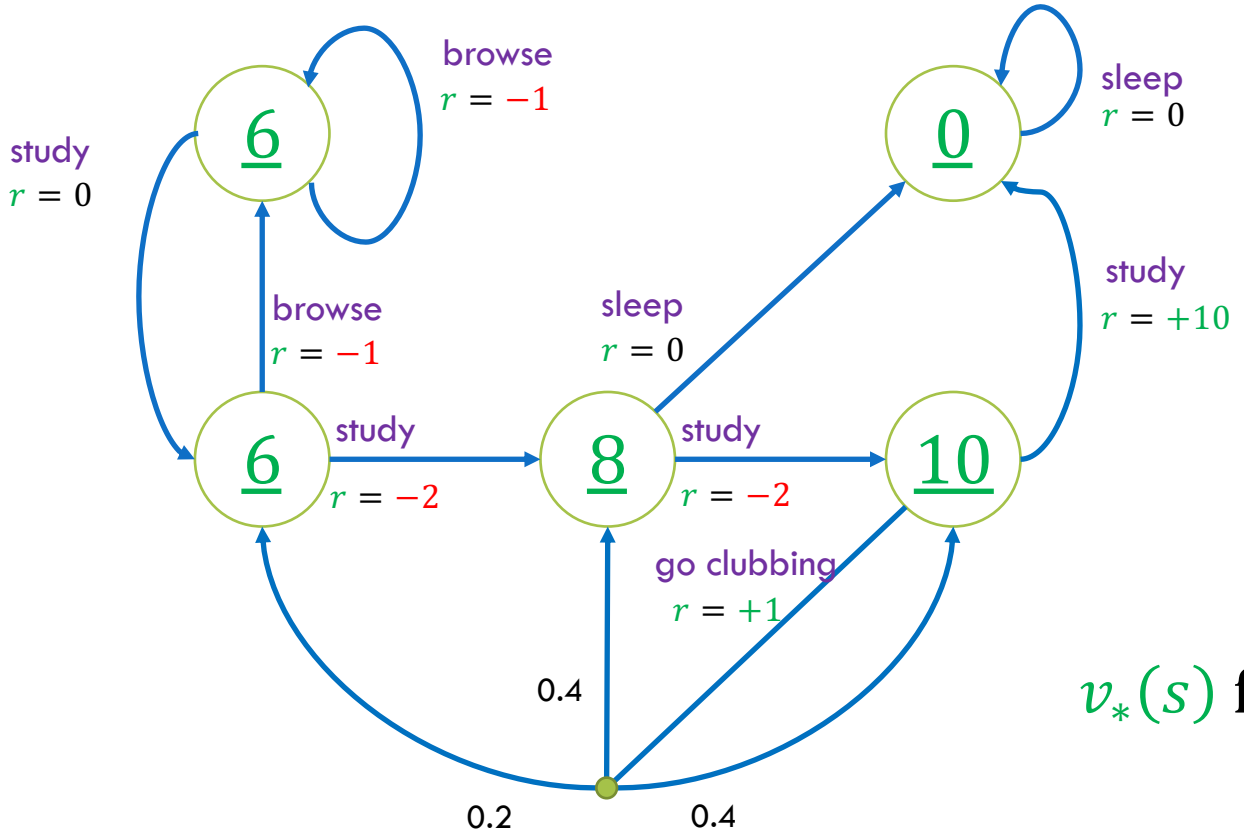
$$v_*(s) = \max_{\pi} v_\pi(s)$$

The optimal action-value function $q_*(s, a)$ is the maximum action–value function over all policies:

$$q_*(s, a) = \max_{\pi} q_\pi(s, a)$$

- These optimal functions specify the best possible performance in the MDP.
- We say that an MDP is solved when we know the optimal value functions.

Optimum Value Function for Student MDP



$v_*(s)$ for $\gamma = 1$

Optimal Policy

Define a partial ordering over the possible policies:

$$\pi \geq \pi' \qquad \text{if} \qquad v_\pi(s) \geq v_{\pi'}(s), \forall s$$

<u>Theorem</u>: For any MDP,

- There exists an optimal policy π_* that is better than or equal to all other policies, $\pi_* \geq \pi, \forall \pi$
- All optimal policies achieve the optimal value function $v_{\pi_*}(s) = v_*(s)$
- They also achieve the optimal action–value function $q_{\pi_*}(s, a) = q_*(s, a)$

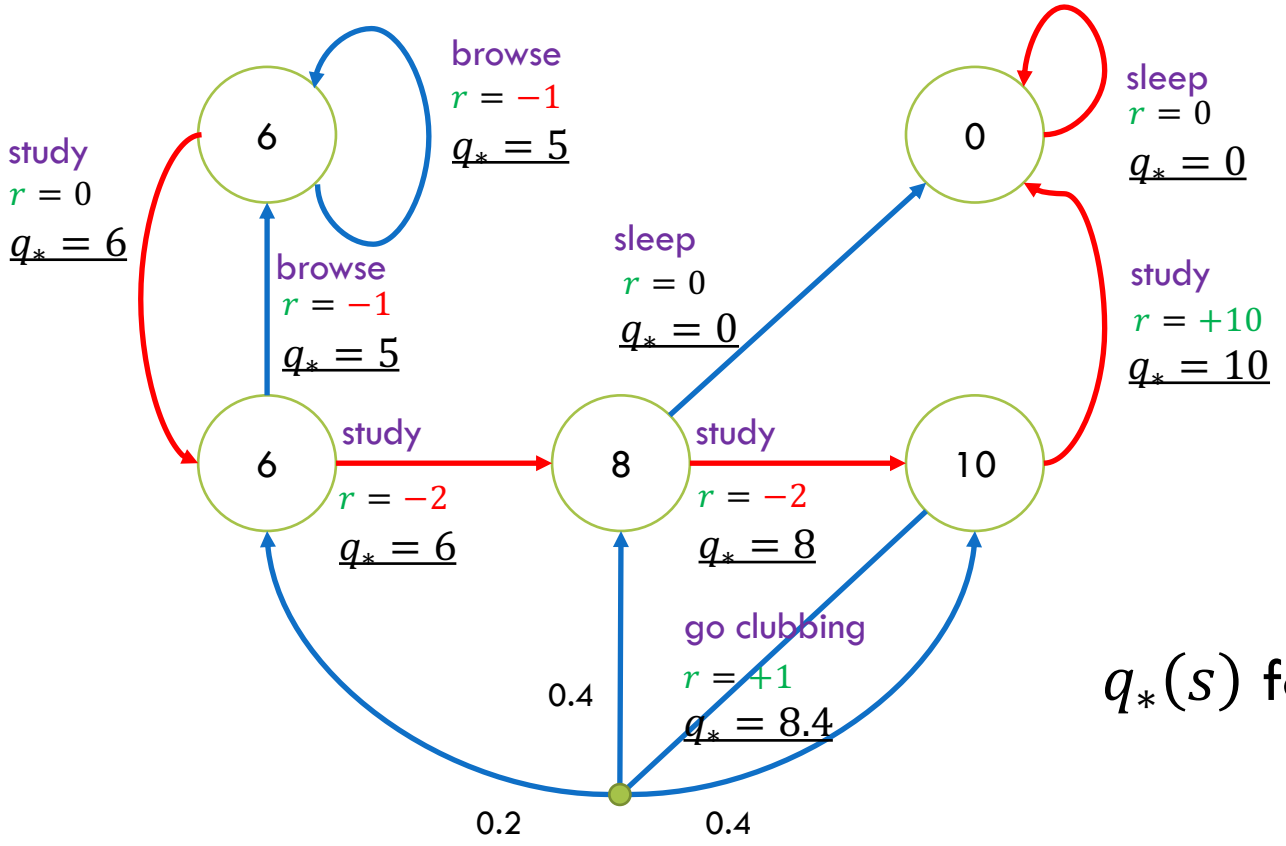
Finding an Optimal Policy

An optimal policy can be found by maximising over $q_*(s, a)$

$$\pi_*(a|s) = \begin{cases} 1 & \text{if } a = \underset{a \in \mathcal{A}}{\operatorname{argmax}}\, q_*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

- There is always a deterministic optimal policy for any MDP
- If we know $q_*(s, a)$, we immediately have the optimal policy

Student Action–Value Function for Student MDP



$q_*(s)$ for $\gamma = 1$

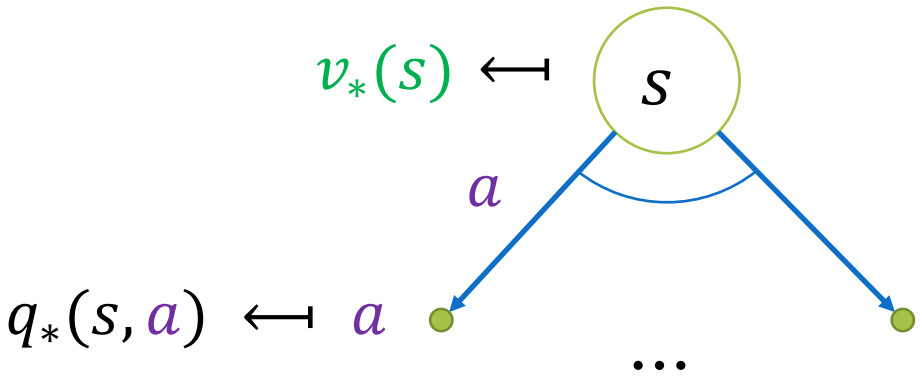
Bellman Optimality

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

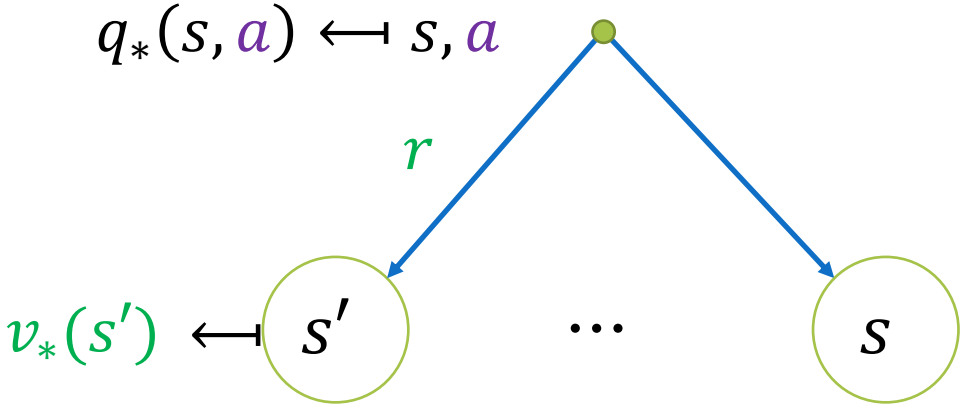
Bellman Optimality Equation for V^*

The optimal value functions are recursively related by the Bellman optimality equations:



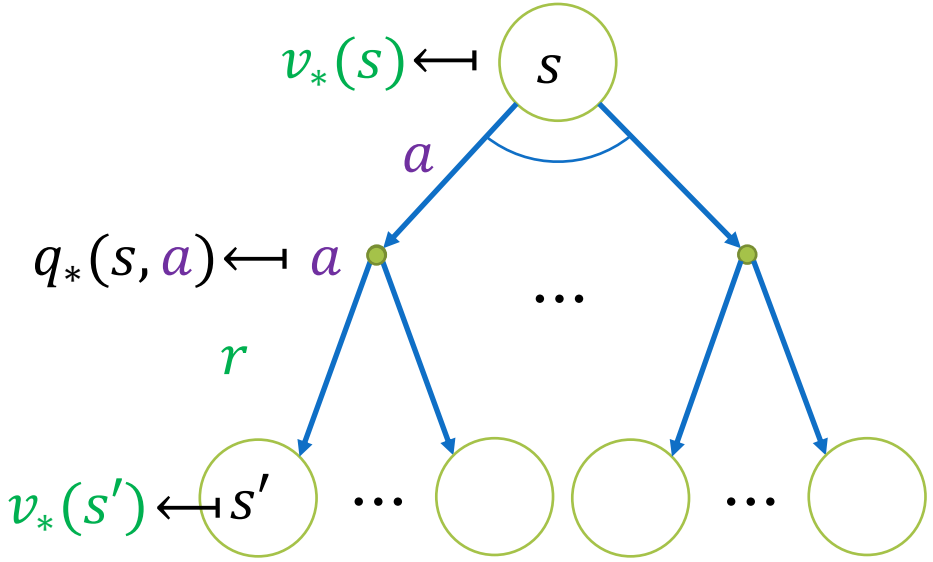
$$v_*(s) = \max_a q_*(s,a)$$

Bellman Optimality Equation for Q^*



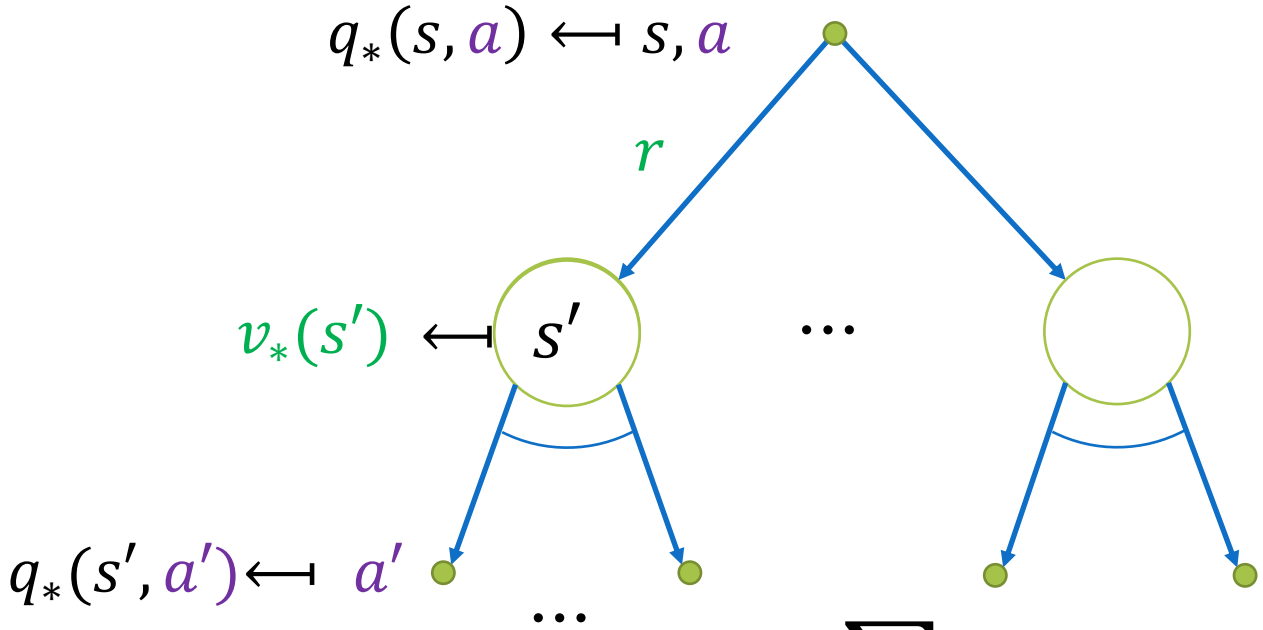
$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{s,s'}^a \, v_*(s')$$

Recursive Formulation for V^*



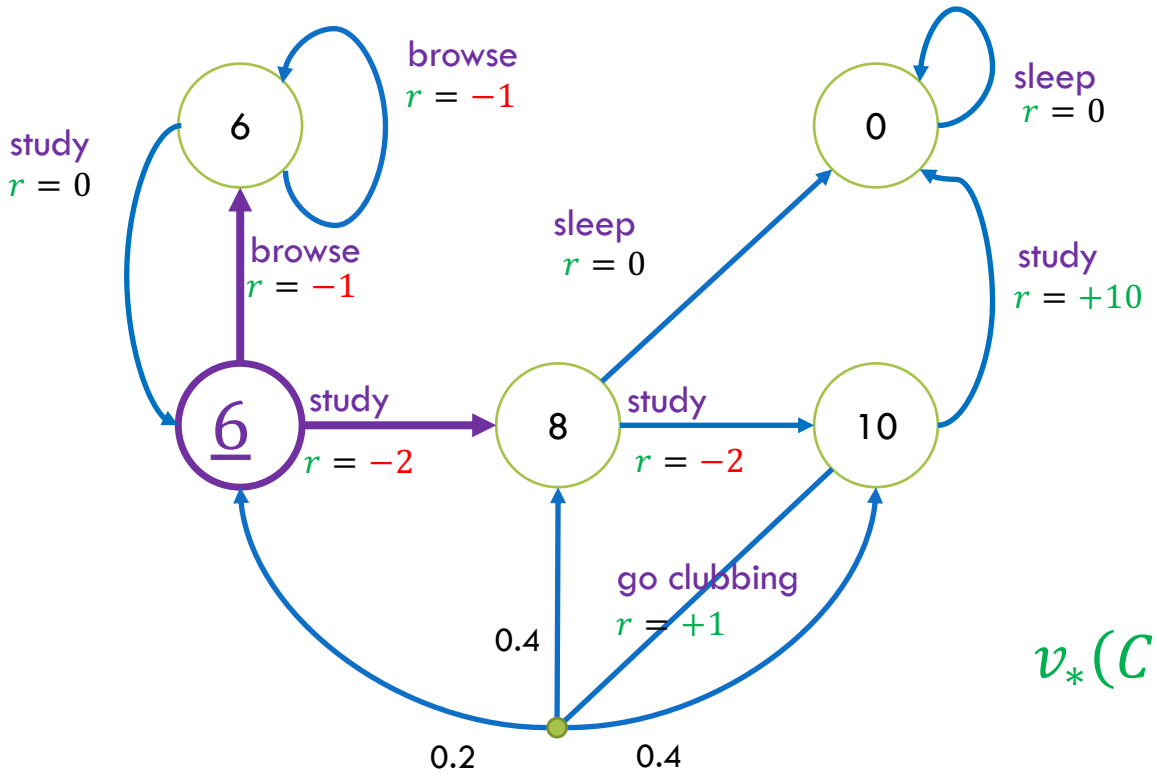
$$v_*(s) = \max_a \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{s,s'}^a \, v_*(s')$$

Recursive Formulation for Q^*



$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{s,s'}^a \max_{a'} q_*(s', a')$$

Example Optimality Equation in Student MDP



$$v_*(C1) = \max(-2 + 8, -1 + 6)$$
$$= 6$$

Solving Bellman Optimality Equations

The Bellman Optimality Equation is non-linear

- Why? Due to the $\max$ operations
- No closed form solution (in general)

Many iterative solution methods

- Policy Iteration
- Value Iteration
- Q-learning

Policy and Value Iteration

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Policy Evaluation

Compute state–value function V^{π} for an arbitrary policy π:

$$
\begin{aligned}
V^{\pi}(s) &= \mathbb{E}_{\pi}\{r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ \ldots | s_t = s\} \\
&= \mathbb{E}_{\pi}\{r_{t+1} + \gamma V^{\pi}(s_{t+1}) | s_t = s\} \\
&= \sum_a \pi(s,a) \sum_{s'} \mathcal{P}^a_{s,s'} \left[\mathcal{R}^a_{s,s'} + \gamma V^{\pi}(s')\right]
\end{aligned}
$$

Why do we want this?

- To evaluate the goodness of a policy

Policy Evaluation

<u>inputs</u>: mdp, an MDP with states $\mathcal{S}$, transitional model $\mathcal{P}$, reward function $\mathcal{R}$, discount γ
π, the policy to be evaluated
<u>local variables</u>: V, a vector of values for states in $\mathcal{S}$, initially zero
Δ, the maximum change in value of any state in an iteration

repeat

$\Delta \leftarrow 0$

for each state $s \in \mathcal{S}$ **:**

$v \leftarrow V(s)$
$V(s) \leftarrow \sum_a \pi(s,a) \sum_{s'} \mathcal{P}^a_{s,s'} \left[\mathcal{R}^a_{s,s'} + \gamma V(s')\right]$
$\Delta \leftarrow \max\left(\Delta, |v - V(s)|\right)$

until $\Delta < \epsilon$

return $V \cong V^\pi$

Policy Improvement

After evaluating a policy, how do we make it better?

What if we take some other action $a \neq \pi(s)$ at state s and the result is better than $V^{\pi}(s)$?

…Then we need to change our policy!

Extend this to consider changes at all states and all possible actions.

Select at each state the action that appears best according to $Q^{\pi}(s, a)$

Policy Iteration

Outer Loop

Inner Loop

1. Initialization
   $V(s) \in \Re$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Repeat
   $\Delta \leftarrow 0$
   For each $s \in \mathcal{S}$:
   $v \leftarrow V(s)$
   $V(s) \leftarrow \sum_{s'} \mathcal{P}_{ss'}^{\pi(s)} \left[\mathcal{R}_{ss'}^{\pi(s)} + \gamma V(s') \right]$
   $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number)

3. Policy Improvement
   policy-stable $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
   $b \leftarrow \pi(s)$
   $\pi(s) \leftarrow \arg\max_a \sum_{s'} \mathcal{P}_{ss'}^{a} \left[\mathcal{R}_{ss'}^{a} + \gamma V(s') \right]$
   If $b \neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop; else go to 2

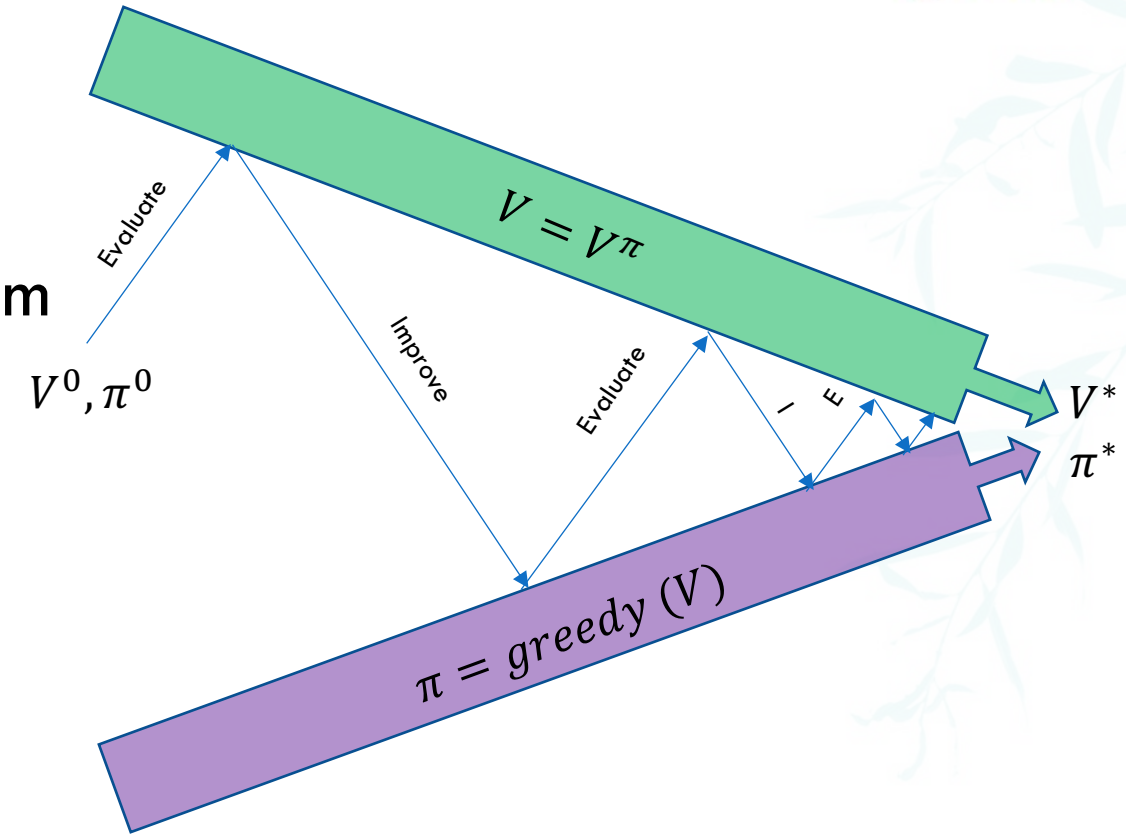
Basic Idea:

Iterate over policies until some stable policy is reached.

Generalised Policy Iteration

Start with a random value function and policy

And iteratively improve them



Our goal: We want π^*.

To achieve this, does policy evaluation need to converge to V^{π}?

Why not update the policy every iteration? → Value iteration.

Value Iteration

In Policy Iteration, each iteration involves a complete policy evaluation, which may itself be a protracted iterative computation, requiring multiple sweeps through the state space.

Instead use Value Iteration: Calculate the return of each state, then use those state returns to find the optimal policy;
i.e., use a truncated form of Policy Evaluation

Value Iteration

<u>inputs</u>: mdp, an MDP with states $\mathcal{S}$, transitional model $\mathcal{P}$, reward function $\mathcal{R}$, discount γ
ϵ, the maximum error allowed in the utility of any state
<u>local variables</u>: V, a vector of values for states in $\mathcal{S}$, initially zero
Δ, the maximum change in value of any state in an iteration

repeat

$\Delta \leftarrow 0$

for each state $s \in \mathcal{S}$ **:**

$v \leftarrow V(s)$
$V(s) \leftarrow \max \sum_{s'} \mathcal{P}^a_{s,s'} \left[\mathcal{R}^a_{s,s'} + \gamma V(s')\right].$
$\Delta \leftarrow \max\left(\Delta, |v - V(s)|\right)$

until $\Delta < \epsilon$

return $V \cong V^*$

… then use V to output the best policy π^*, such that

$$\pi(s) = \arg\max_a \sum_{s'} \mathcal{P}^a_{s,s'} \left[\mathcal{R}^a_{s,s'} + \gamma V(s')\right]$$

Q Learning

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

The optimal policy

We want to choose a value-maximizing policy.

However, we are missing two key bits of information:

1. The rewards of unobserved states.
2. What the next state will be when we take an action.

<u>The challenge</u>:
Infer rewards/state transitions as we go along
…while maximizing revenue.

Markov Decision Process

MDP $\equiv \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

Completely specified by a distribution:

$$\Pr[s_{t+1} = s; r_{t+1} = r | s_t, a_t]$$

"What is the next state and reward given current state and action?"

<u>Planning</u>: given an MDP, compute the optimal policy.

<u>Learning</u>: don't know the MDP, learn a strategy.

Markov Decision Problem

Value / Policy Iteration solves Planning

(we are model based, we know the MDP)

But… in learning, we don't know the underlying MDP.

We need to perform trial-and-error, interact with environment and not lose too much reward along the way.

Q Learning: iteration, redux

Learn this

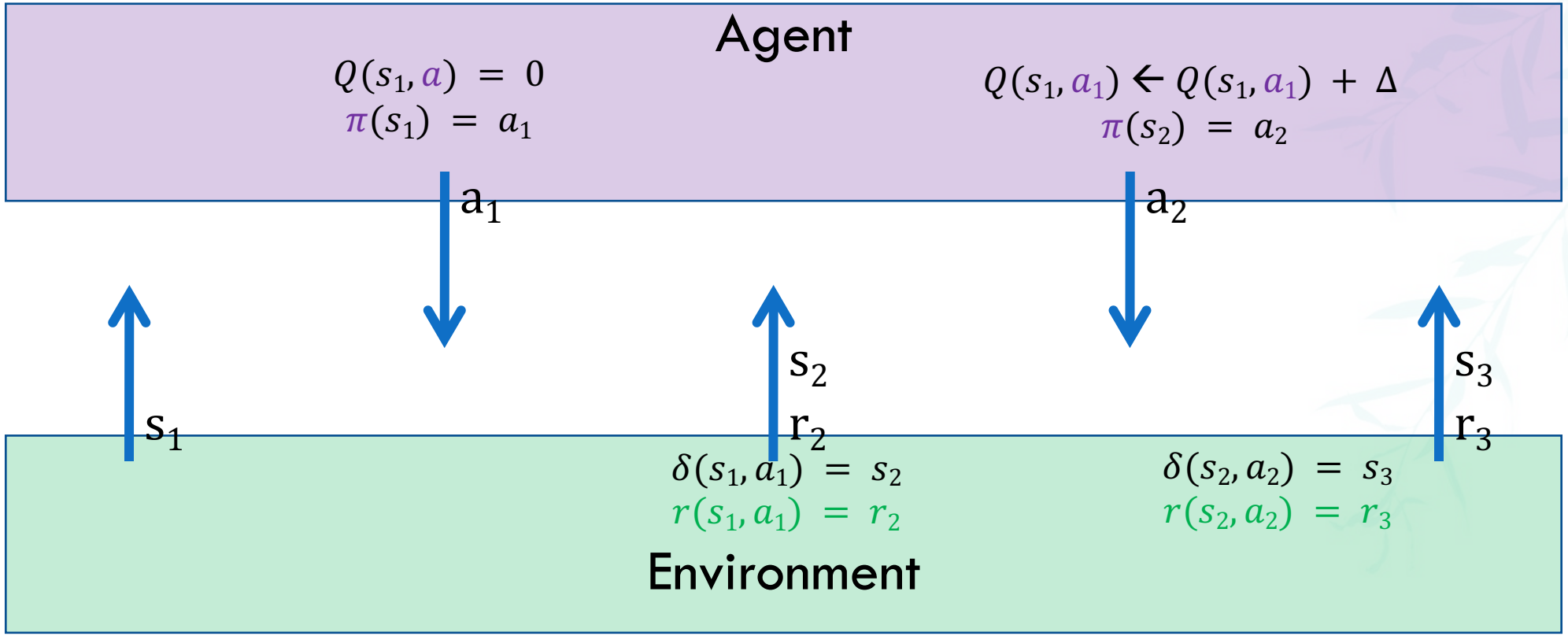
… and this

$$q_*(s,a) = \cancel{\mathcal{R}_s^a} r(s,a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{s,s'}^a \max_{a'} q_*(\cancel{s', a'}\ \delta(s,a))$$

We don't have the optimal Q function Q_* at first …

- Let's act as if we do
- Updates will bring it closer to optimal … until it is optimal!

Q-Learning: The Procedure



Q Learning

<u>local variables</u>: $\hat{Q}$, a vector of Q values for state action pairs in $\mathcal{S} \times \mathcal{A}$, initially zero

$s \leftarrow s_0$

repeat

select action a

receive reward $r(s,a)$

$s' \leftarrow \delta(s,a)$

$\hat{Q}(s,a) \leftarrow r(s,a) + \gamma \max \hat{Q}(s',a')$

Key observation: When $r(s,a) \geq 0$ and $\hat{Q} = 0, \hat{Q}(s,a) \leq Q^*(s,a)$ always.

In other words – we always underestimate the optimal Q values.

They always increase at every iteration, thus we converge to Q^*… and in particular to an optimal policy!

Handling Non-determinism

local variables: $\hat{Q}$, a vector of Q values for state action pairs in $\mathcal{S}\times\mathcal{A}$, initially zero

$s \leftarrow s_0$
repeat
select action a
receive reward $r(s,a)$
$s' \leftarrow \delta(s,a)$

$$\hat{Q}(s,a) \leftarrow r(s,a) + \gamma \max \hat{Q}(s',a')$$

Problem 1: both $r(s,a)$ and $r(s,a)$ are stochastic. We should factor current $\hat{Q}(s,a)$ in the computation instead of replacing it entirely.

Solution: update $\hat{Q}(s,a)$ in a gradual manner, as we observe it more:

$$\hat{Q}(s,a) \leftarrow (1-\alpha)\cdot\left[\hat{Q}(s,a)\right] + \alpha \cdot \left[r(s,a) + \gamma \max \hat{Q}(s',a')\right], \text{ where } \alpha = \frac{1}{1+N[s,a]}$$

# of times a taken at state s

Exploration vs. Exploitation

local variables: $\hat{Q}$, a vector of Q values for state action pairs in $\mathcal{S} \times \mathcal{A}$, initially zero

$s \leftarrow s_0$
repeat

select action a
receive reward $r(s, a)$
$s' \leftarrow \delta(s, a)$
$\hat{Q}(s, a) \leftarrow r(s, a) + \gamma \max \hat{Q}(s', a')$

Problem 2: How do we select an action a? Greedily? Then we could miss out better actions.

First Cut Solution: pick action a greedy, but at random some small percentage of time (a.k.a. ϵ-greedy)

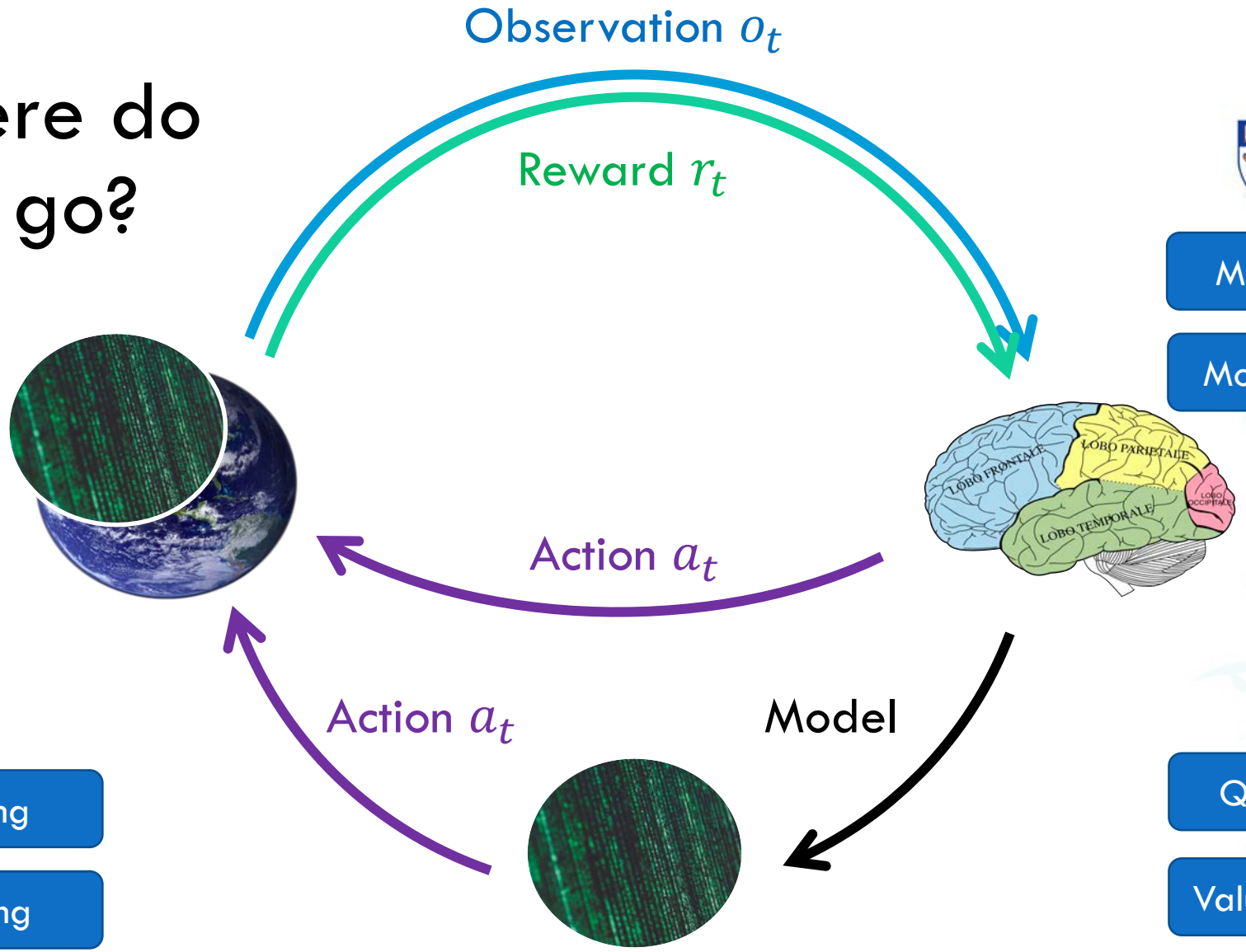
Your Turn: Can you suggest a better method for choosing action a?

Summing Up: Planning versus Learning

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Where do they go?



Model Free

Model Based

Q Learning

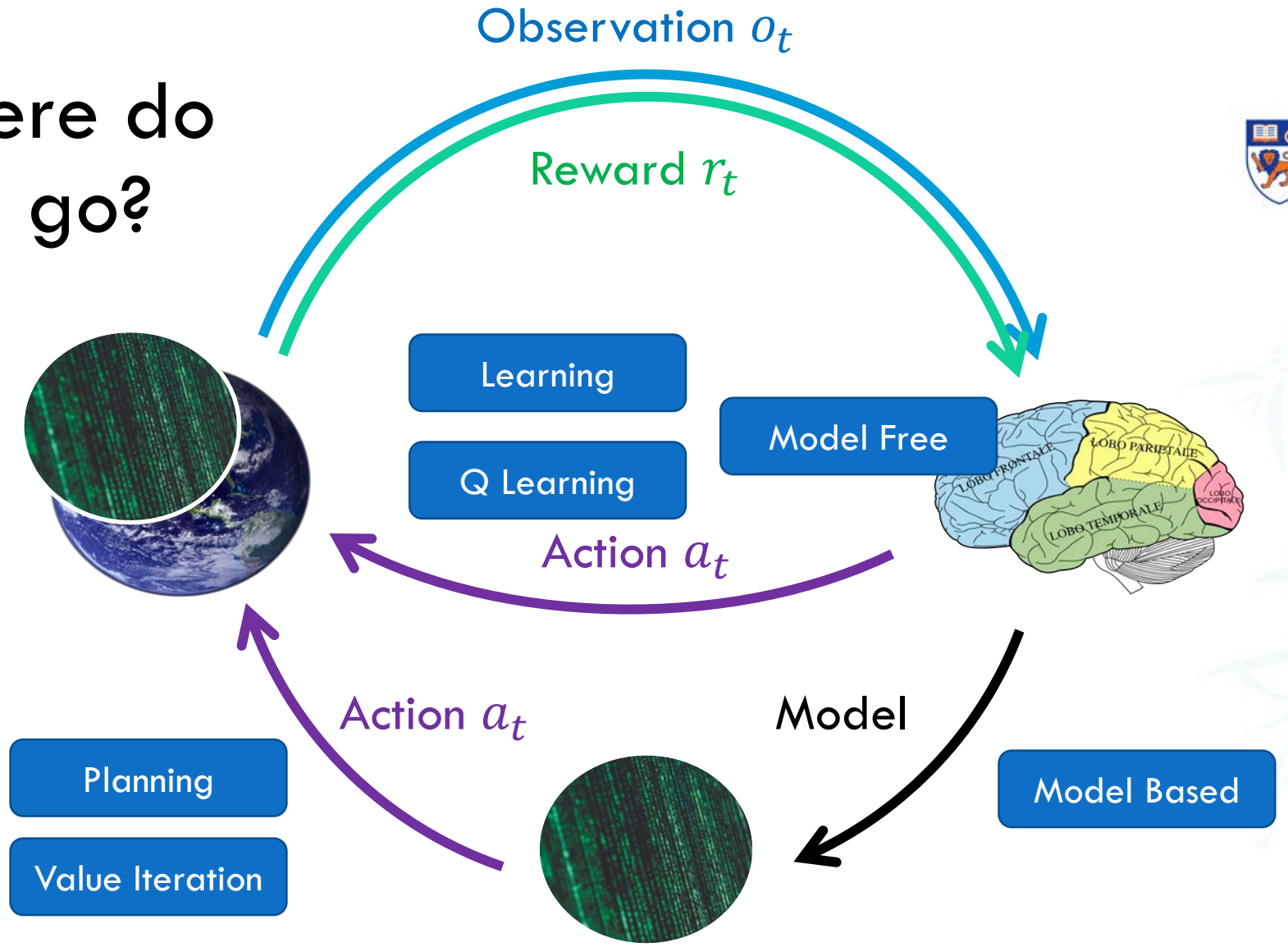
Value Iteration

Planning

Learning

Action a_t

Where do they go?



Action a_t